Open in Colab

After clicking the "Open in Colab" link, copy the notebook to your own Google Drive before getting started, or it will not save your work

BYU CS 180 Lab 4: Data Wrangling

Introduction:

In this lab, we will analyze college football data from the years 2016-2020. The data is spread across multiple files and will require a bit of “data wrangling”.

Once we have cleaned and processed the data, we will put our statistics knowledge to good use by digging a little deeper than simple summary statistics.

Very rarely are data scientists handed a pristine data set ready for analysis. More often than not, quite a bit of work is required to clean and preprocess the data so that it's ready for analysis.

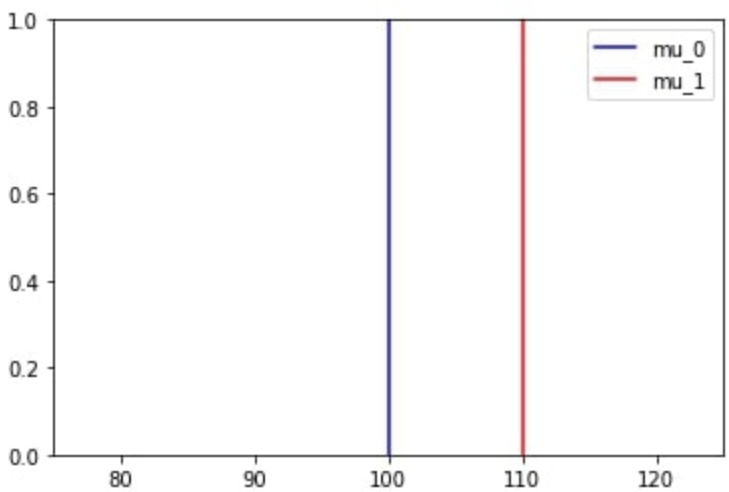
Data wrangling, also called **data cleaning**, **data remediation**, or **data munging**, refers to a variety of processes designed to transform raw data into more readily usable formats. The exact methods differ from project to project depending on the data you're leveraging and the goal you're trying to achieve.

Some examples of data wrangling include:

- Merging multiple data sources into a single dataset for analysis.
- Identifying gaps in data (for example, empty cells in a spreadsheet) and either filling or deleting them.
- Deleting data that's either unnecessary or irrelevant to the project you're working on.
- Identifying extreme outliers in data and either explaining the discrepancies or removing them.

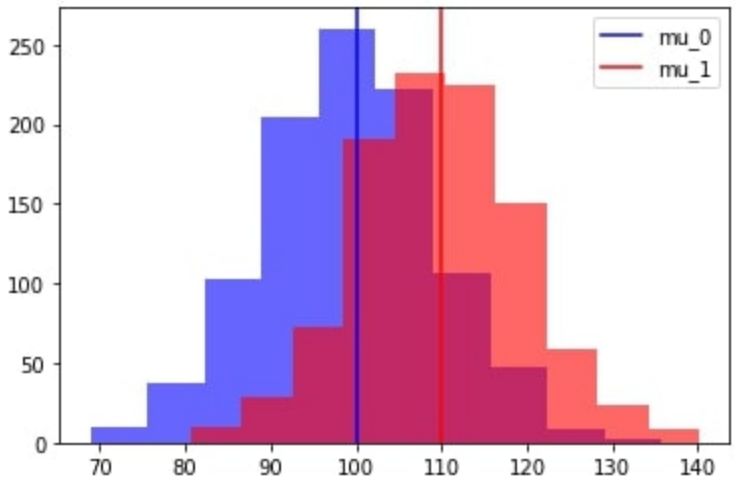
Measures of Variability

We are very used to comparing point estimates. For example, in order to tell if one thing is better than the other, we may look at the average of each over time. Consider the following plot comparing the average of two groups, mu_0 and mu_1.



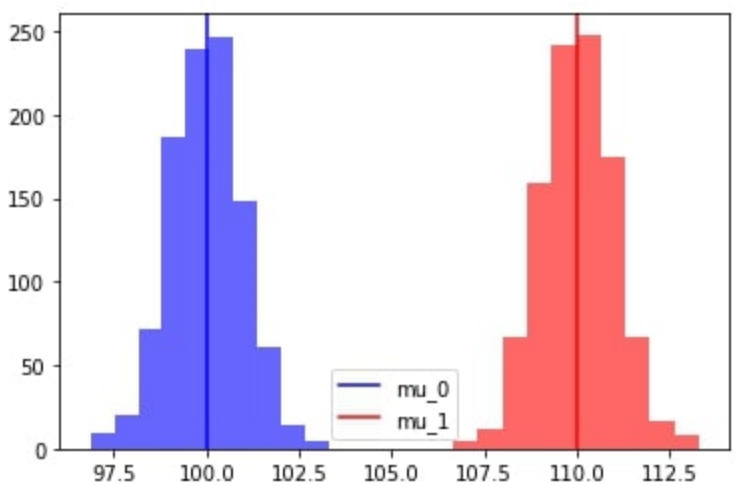
In this case, we might conclude that mu_1 is greater because it's average is higher. However, we are ignoring an important aspect of the data: its variability.

When we plot the **variability and the mean**, we observe the following plot:



Now the distributions of mu_1 and mu_0 are so close that it's difficult to say with certainty that mu_1 is better. For any random draw of mu_1, it's roughly a 50-50 chance to actually be greater than a random draw of mu_0.

Now suppose we observe the following plot:



While the means are the same as before, the variability of the two distributions are now very different. We can say with a high degree of certainty that mu_1 is higher.

Keep this idea in mind as you analyze the dataset for this lab.

Data:

Data from the 2016, 2017, 2018, 2019, and 2020 college football seasons are also available on the course GitHub:

```
# Download the data located at the following URLs
cfb16_url = "https://raw.githubusercontent.com/rhodes-byu/cs180-winter25/refs/heads/
cfb17_url = "https://raw.githubusercontent.com/rhodes-byu/cs180-winter25/refs/heads/
cfb18_url = "https://raw.githubusercontent.com/rhodes-byu/cs180-winter25/refs/heads/
cfb19_url = "https://raw.githubusercontent.com/rhodes-byu/cs180-winter25/refs/heads/
cfb20_url = "https://raw.githubusercontent.com/rhodes-byu/cs180-winter25/refs/heads/
```

Exercise 1: Yearly Counts

Exercise Question

Read in the files, and add a year column to each file (from the original .csv file name).

```
import pandas as pd
```

```
# Write your code to read in the files and add the year values from each csv:
cfb16 = pd.read_csv(cfb16_url)
cfb17_url = pd.read_csv(cfb17_url)
cfb18_url = pd.read_csv(cfb18_url)
cfb19_url = pd.read_csv(cfb19_url)
cfb20_url = pd.read_csv(cfb20_url)

cfb16['Year'] = 2016
cfb17_url['Year'] = 2017
cfb18_url['Year'] = 2018
cfb19_url['Year'] = 2019
cfb20_url['Year'] = 2020
```

Exercise 2: Data Aggregation

Exercise Question

Combine every file into a single dataframe.

```
# Write your code to combine all of the csvs into one dataframe here:
new_data = pd.concat([cfb16, cfb17_url, cfb18_url, cfb19_url, cfb20_url])
```

```
new_data
```

	Team	Games	Win	Loss	Off.Rank	Off.Plays	Off.Yards	Off.Yards.Play
0	Akron (MAC)	12	5	7	84	776	4649	5.99
1	Alabama (SEC)	15	14	1	34	1056	6829	6.47
2	Appalachian St. (Sun Belt)	13	10	3	52	912	5589	6.13
3	Arizona (Pac-12)	12	3	9	67	815	4957	6.08
4	Arizona St. (Pac-12)	12	5	7	81	900	4689	5.21
...	...	...	...	...	...	...	...	...
122	West Virginia (Big 12)	9	5	4	42	690	3804	5.51
123	Western Ky. (C-USA)	11	5	6	120	699	3200	4.58
124	Western Mich. (MAC)	6	4	2	15	392	2878	7.34
125	Wisconsin (Big Ten)	6	3	3	93	431	2153	5.00
126	Wyoming (Mountain West)	6	2	4	82	409	2237	5.47

631 rows × 153 columns

Exercise 3: Conference Search

Exercise Question

Create a `conference` field by parsing the `team` column.

Example:

Team	Conference
Penn State University (Big Ten)	Big Ten

```
# Write your code to parse the conference from the team name:
new_data['Conference'] = new_data['Team'].str.extract(r'\((.*?)\)')
```

```
new_data['Conference'].unique()
```

```
array(['MAC', 'SEC', 'Sun Belt', 'Pac-12', 'FBS Independent', 'Big 12',
       'ACC', 'C-USA', 'AAC', 'Big Ten', 'FL', 'OH', 'MWC', 'Independent',
       'Mountain West'], dtype=object)
```

Exercise 4: Big Ten Vs. South Eastern

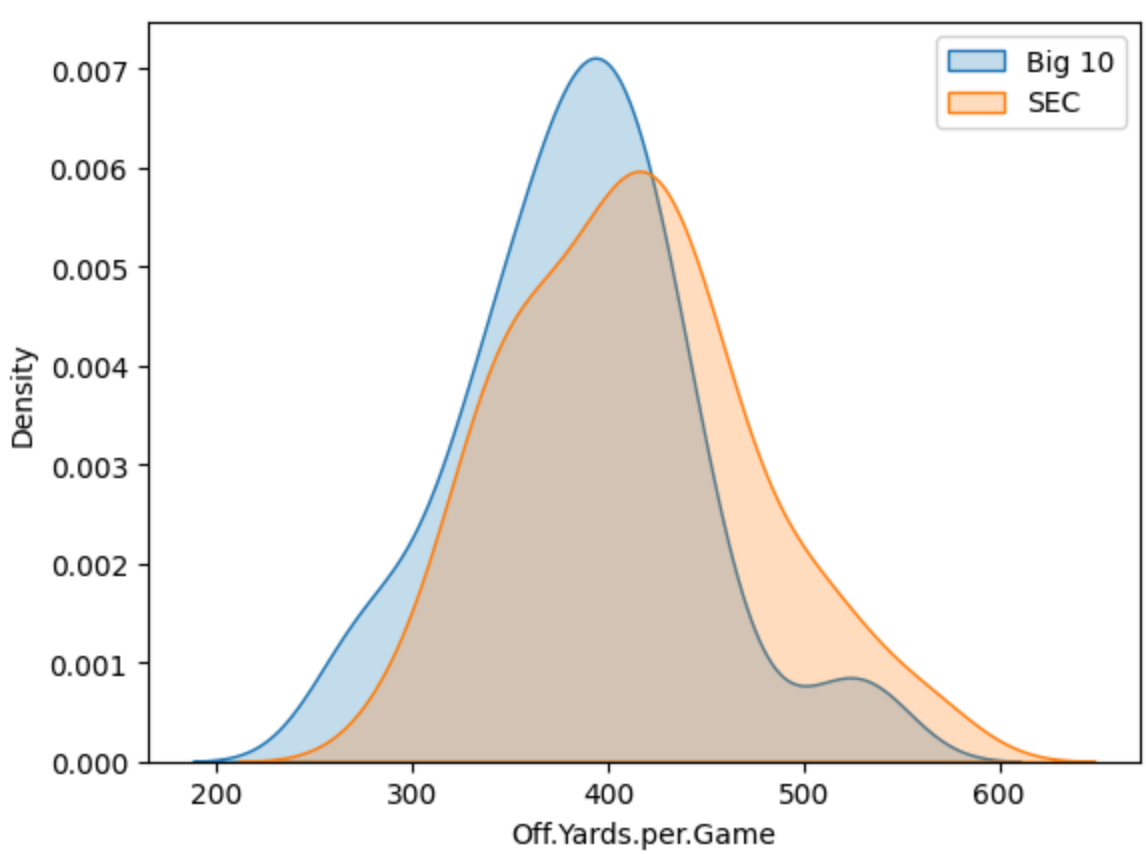
Exercise Question 4a: Offense

- Is there a statistical difference between the Big Ten Conference and the South Eastern Conference in terms of `Off.Yards.per.Game`? Use a seaborn KDE plot to create a figure. Comment on the difference in means and the overlap of distributions.
- Do the same as above for `Off.TDs`.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
Big_10 = new_data[new_data['Conference'] == 'Big Ten']
SEC = new_data[new_data['Conference'] == 'SEC']
```

```
# Write the code for the statistical differences for Off.Yards.per.Game:
sns.kdeplot(data=Big_10, x='Off.Yards.per.Game', fill=True, label = "Big 10")
sns.kdeplot(data=SEC, x='Off.Yards.per.Game', fill=True, label = "SEC")
plt.legend()
plt.show()
```




```
Big_10['Off.Yards.per.Game'].mean()
```

```
385.5642857142857
```

```
SEC['Off.Yards.per.Game'].mean()
```

```
412.7885714285715
```

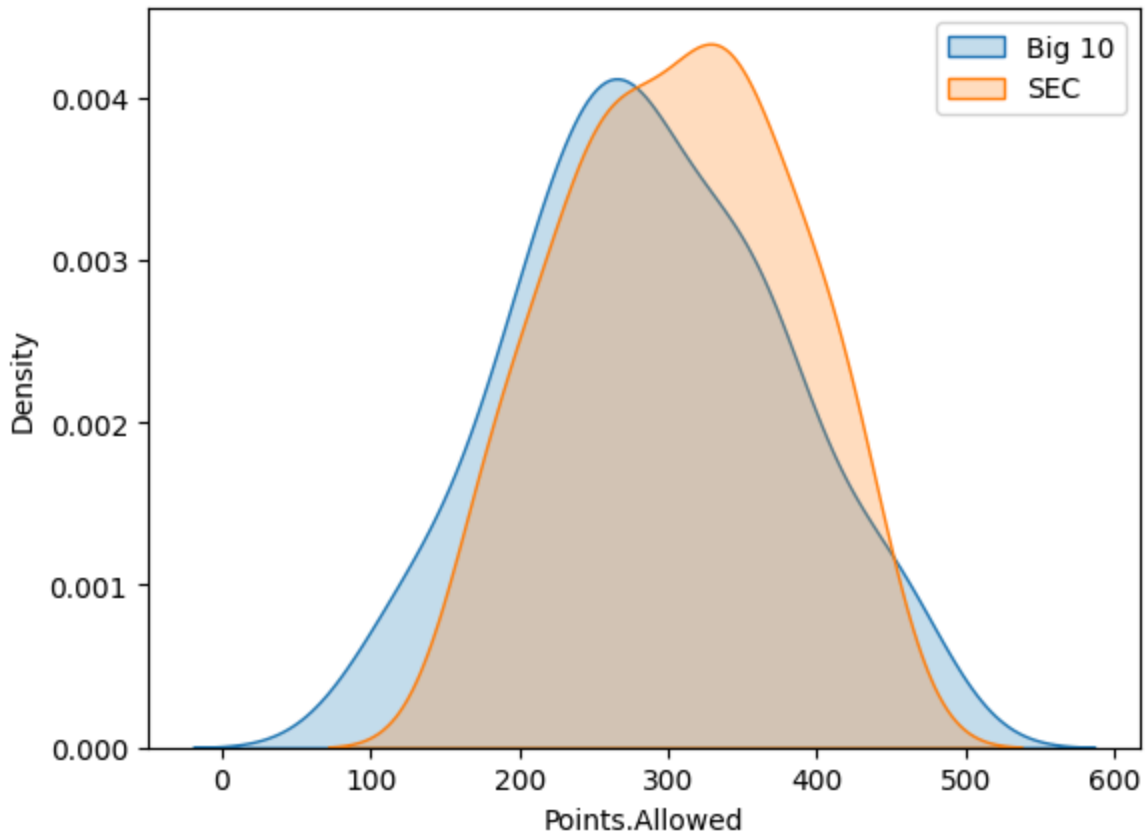
(Comment on the difference in means and the overlap of distributions here)

The majority of the data overlaps, the mean of the SEC is higher than the mean of the Big 10 meaning that on average in the SEC there are more yards per game.

Exercise Question 4b: Defense

- Is there a statistical difference between the Big Ten Conference and the South Eastern Conference in terms of `Points.Allowed`? Use a seaborn KDE plot to create a figure. Comment on the difference in means and the overlap of distributions.
- Do the same as above for `Opp.Pass.Yds.Allowed` and `Opp.Rush.Yards.Allowed`.

```
# Write the code for the statistical differences for Points.Allowed:
sns.kdeplot(data=Big_10, x='Points.Allowed',fill=True, label = "Big 10")
sns.kdeplot(data=SEC,x='Points.Allowed',fill = True, label = "SEC")
plt.legend()
plt.show()
```




```
Big_10['Points.Allowed'].mean()
```

```
286.3285714285714
```

```
SEC['Points.Allowed'].mean()
```

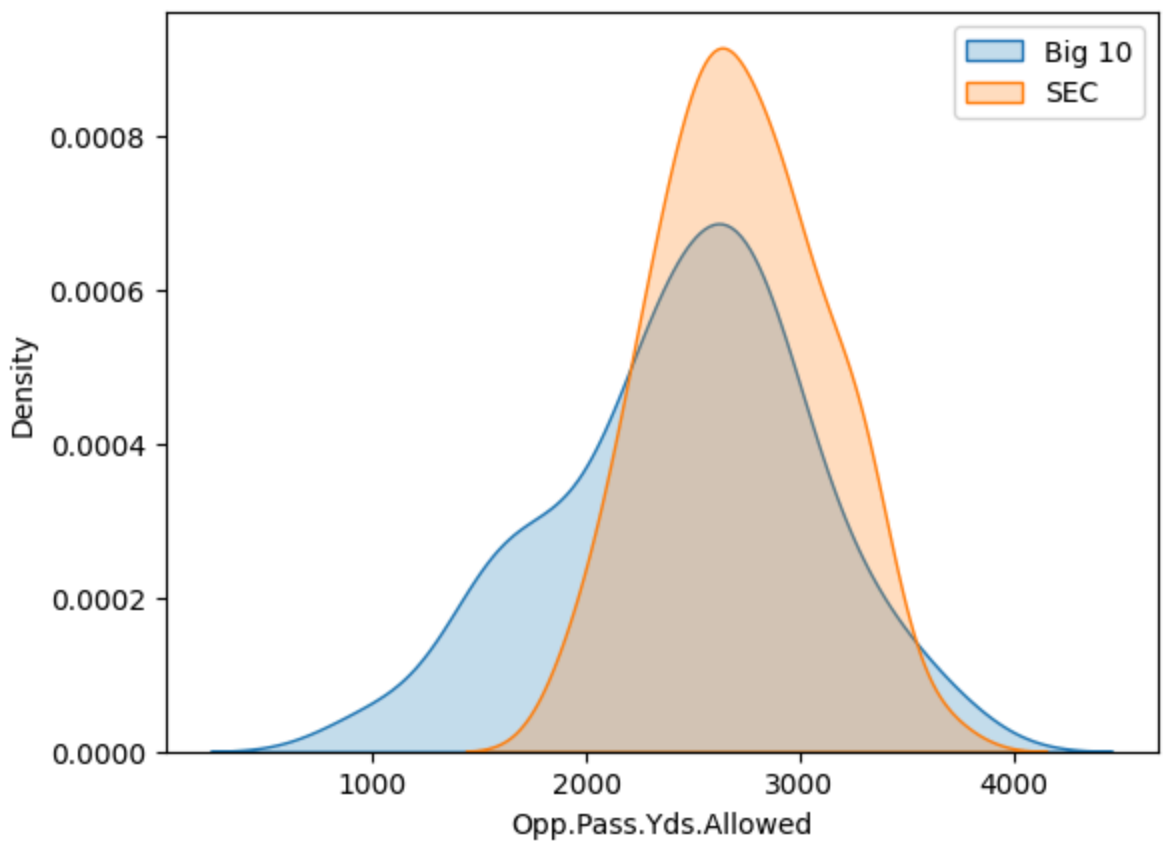
```
306.12857142857143
```

(Comment on the difference in means and the overlap of distributions here)

The data for the Big 10 spans from 0 to ~600 while the data for the SEC spans from ~85 to ~550. There is general overlap between the two. The mean of points allowed per game was higher in the SEC than in the Big 10 meaning that on average there are more points scored per game in the SEC than in the big 10.

```
# Write the code for the statistical differences for Opp.Pass.Yds.Allowed:
sns.kdeplot(data=Big_10, x='Opp.Pass.Yds.Allowed',label = "Big 10",fill=True)
```

```
sns.kdeplot(data=SEC, x='Opp.Pass.Yds.Allowed', label = "SEC",fill=True)
plt.legend()
plt.show()
```




```
Big_10['Opp.Pass.Yds.Allowed'].mean()
```

```
2454.8571428571427
```

```
SEC['Opp.Pass.Yds.Allowed'].mean()
```

```
2716.3
```

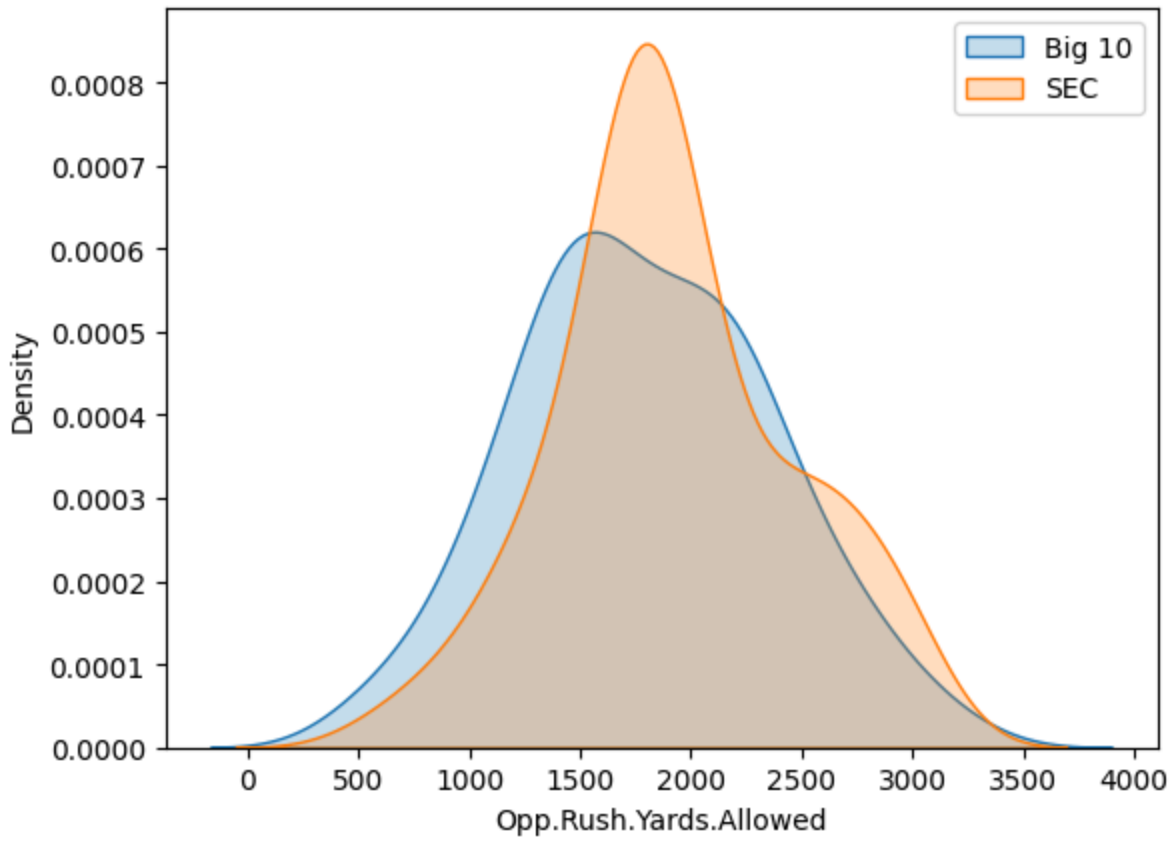
(Comment on the difference in means and the overlap of distributions here)

We see that there is a general overlap of Pass yards allowed by the offense in both the Big 10 and the SEC. The mean however is higher in the SEC than in the Big 10.

Double-click (or enter) to edit

```
# Write the code for the statistical differences for Opp.Rush.Yards.Allowed:
sns.kdeplot(data=Big_10, x='Opp.Rush.Yards.Allowed',fill = True, label="Big 10")
sns.kdeplot(data=SEC, x='Opp.Rush.Yards.Allowed',fill = True,label ="SEC")
```

```
plt.legend()
plt.show()
```




```
Big_10['Opp.Rush.Yards.Allowed'].mean()
```

```
1794.057142857143
```

```
SEC['Opp.Rush.Yards.Allowed'].mean()
```

```
1920.9714285714285
```

(Comment on the difference in means and the overlap of distributions here)

On average in the Big 10 and the SEC, the rushing yards allowed are fairly similar, the mean is slightly higher in the SEC which means that on average they allow for more rushing yards.

Exercise 5: Offense Statistics

Exercise Question 5a: Offense

Is the offense changing over time? Create some plots showing the average offensive production over time (each year). Include an estimate of the variability in your figures.

Comment on whether any trends you see are likely to be true or spurious.

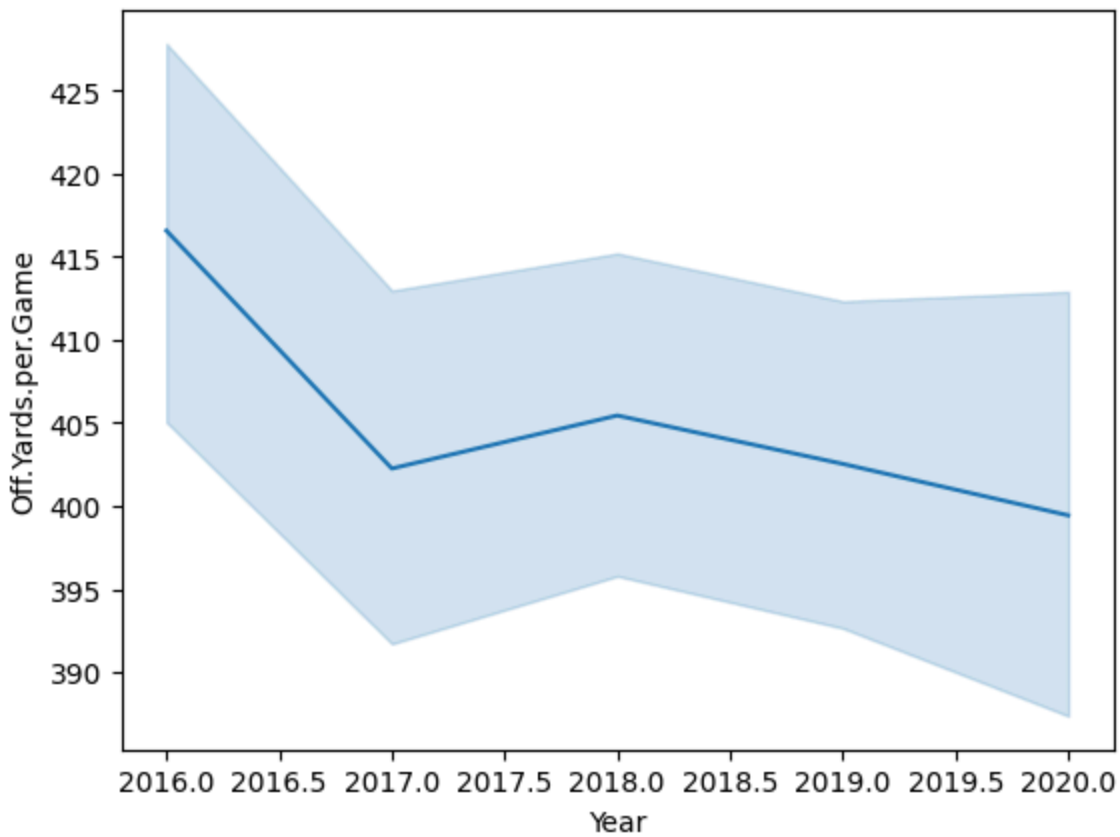
Create a plot for the following metrics:

- `Off.Yards.per.Game`
- `Off.TDs`

```
# Create a few plots showing how each metric changed over time:
data_year = new_data.sort_values(by='Year')
```

```
sns.lineplot(data=data_year, x='Year', y='Off.Yards.per.Game')
```

```
<Axes: xlabel='Year', ylabel='Off.Yards.per.Game'>
```

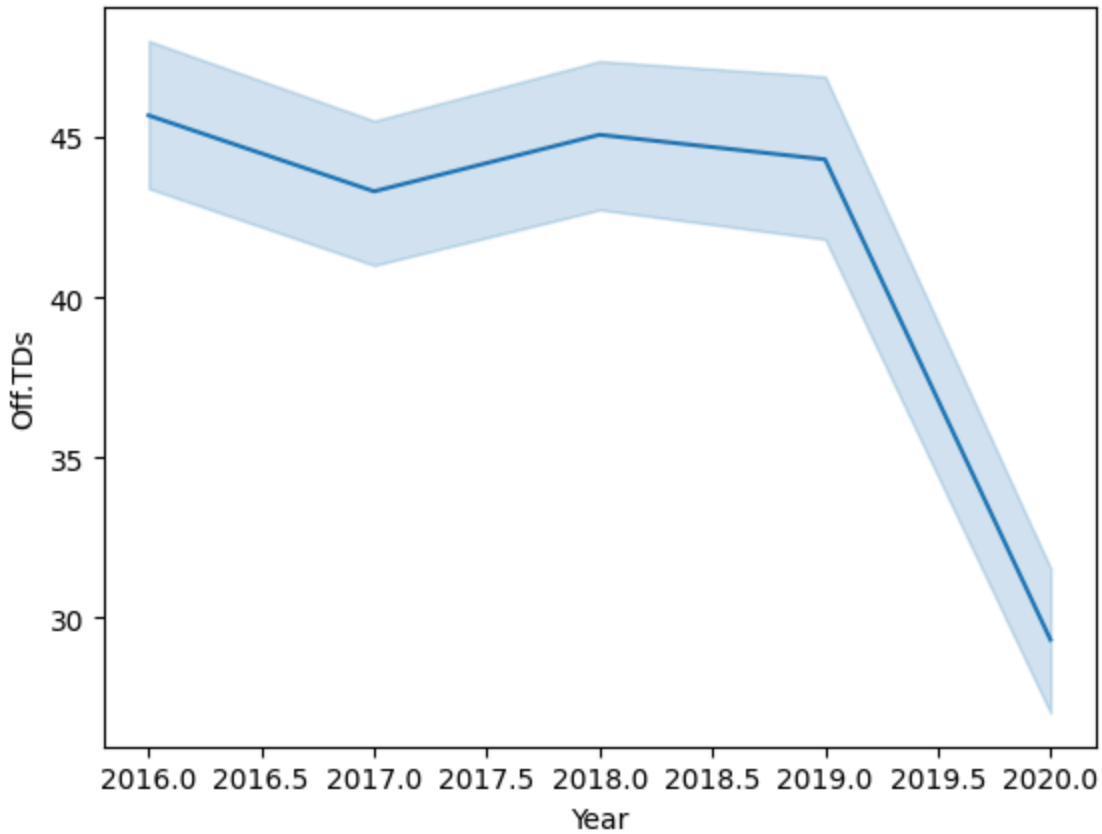


(Comment on any trends you see here)

From our graph we see a general decrease in offensive yards per game from 2016 to the 2020

```
sns.lineplot(data=data_year, x='Year', y='Off.TDs')
```

```
<Axes: xlabel='Year', ylabel='Off.TDs'>
```



We see a decrease in TDs from the year 2016 to 2020. The most dramatic decrease began in ~2019

Exercise Question 5b: Defense

Is the defense changing over time? Create some plots showing the average defensive production over time (each year). Include an estimate of the variability in your figures.

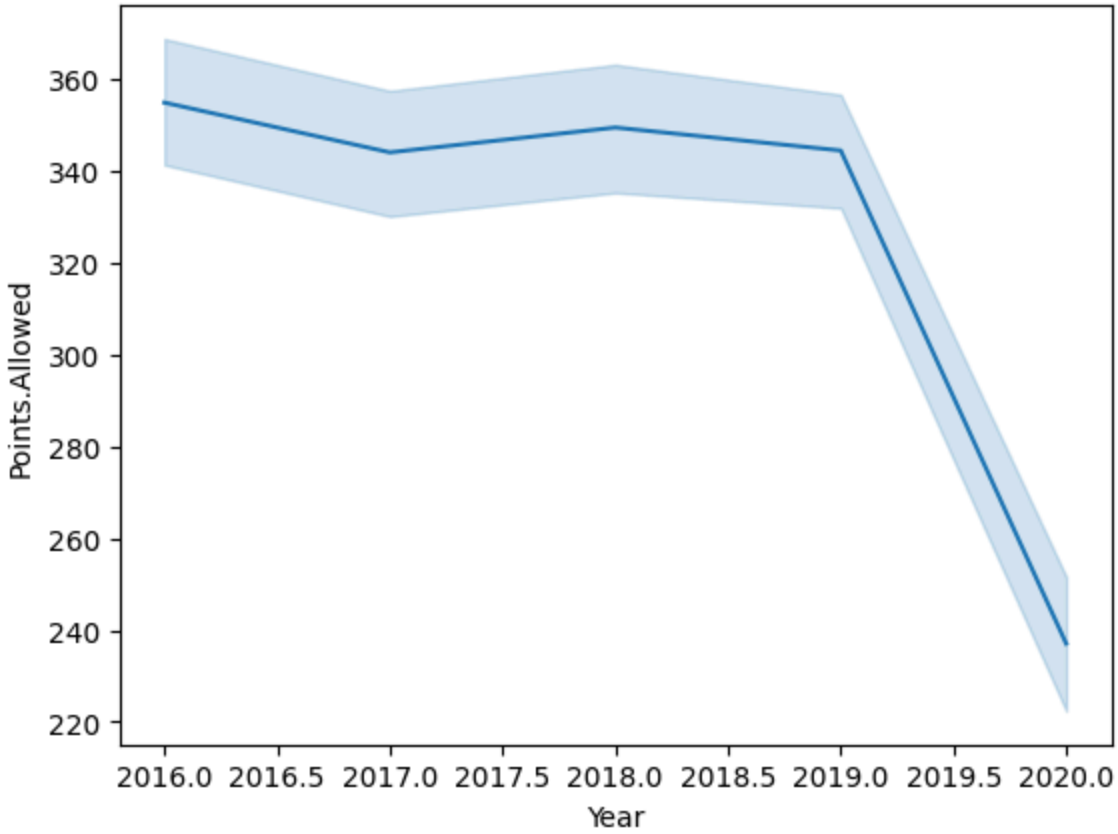
Comment on whether any trends you see are likely to be true or spurious.

Create a plot for the following metrics:

- `Points.Allowed`
- `Opp.Pass.Yds.Allowed`
- `Opp.Rush.Yards.Allowed`

```
# Create a few plots showing how each metric changed over time:
sns.lineplot(data=data_year, x='Year', y='Points.Allowed')
```

```
<Axes: xlabel='Year', ylabel='Points.Allowed'>
```

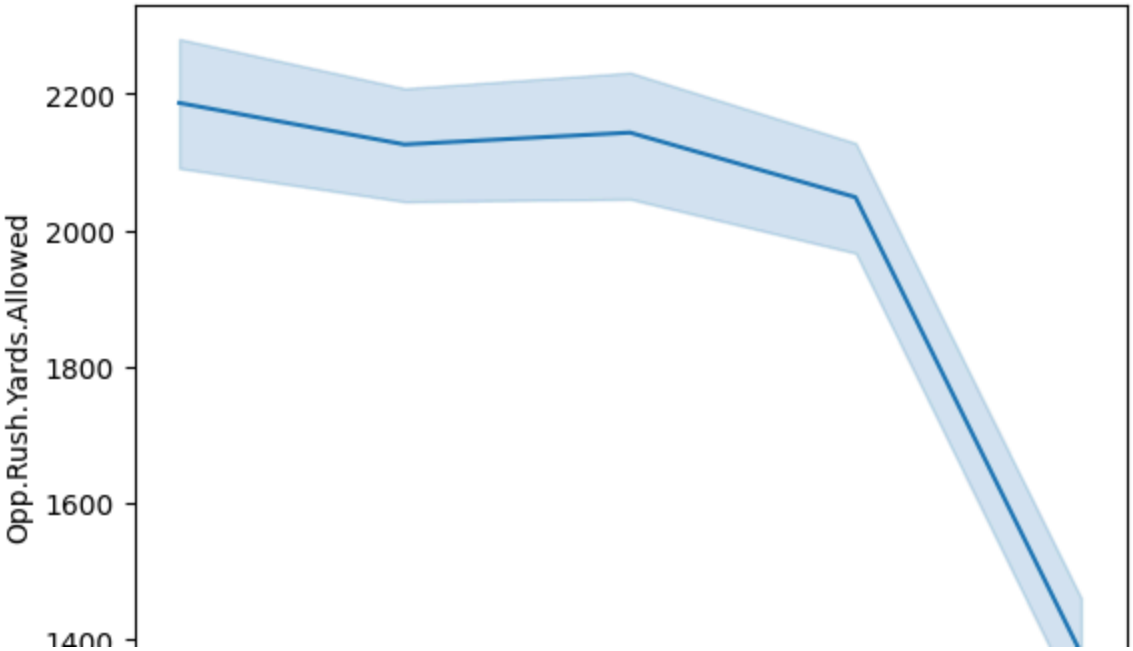


for the range 2016 to 2020, we see an overall decrease in points with the steepest decrease beginning in 2019.

```
sns.lineplot(data=data_year, x='Year',y='Opp.Pass.Yds.Allowed')
```

```
<Axes: xlabel='Year', ylabel='Opp.Pass.Yds.Allowed'>
```

over the range 2016 to 2020, we see a decrease in Pass Yards allowed with the steepest decrease beginning in 2019.

```
sns.lineplot(data=data_year, x='Year',y='Opp.Rush.Yards.Allowed')
```

```
<Axes: xlabel='Year', ylabel='Opp.Rush.Yards.Allowed'>
```